



SISTEMA DE MONITOREO DE HEMOGLOBINA IoT

MANUAL TÉCNICO

1. Descripción General del Sistema

El Sistema de Monitoreo de Hemoglobina IoT es una plataforma médica integral diseñada para la detección temprana de anemia y poliglobulia. Integra un dispositivo de hardware ESP32 con un backend de API REST desarrollado en Laravel (PHP) y un frontend web estático, formando una arquitectura de tres capas.

El sistema recopila mediciones de nivel de hemoglobina (g/dL) y temperatura corporal (°C) desde el sensor conectado al ESP32, las transmite vía HTTP a la API, y las almacena en una base de datos MySQL, generando alertas médicas automáticas mediante triggers y procedimientos almacenados.

2. Arquitectura del Sistema

2.1 Diagrama de Capas

El sistema se compone de tres capas principales:

- Capa de Hardware (IoT): Dispositivo ESP32 con sensores físicos. Dirección MAC registrada: 24:0A:C4:8D:1B:3C.
- Capa de Backend (API REST): Laravel 9 con Sanctum para autenticación por token Bearer.
- Capa de Presentación (Frontend): HTML5 + CSS + JavaScript Vanilla. Archivos: index.html, dashboard.html, estilos.css.

2.2 Stack Tecnológico

Componente	Tecnología	Versión / Detalle
Backend Framework	Laravel	9.x
Lenguaje Backend	PHP	8.x
Autenticación API	Laravel Sanctum	Token Bearer
Base de Datos	MySQL	monitoreo_medico_db
Hardware IoT	ESP32	MAC: 24:0A:C4:8D:1B:3C
Frontend	HTML5 + CSS + JS Vanilla	Sin framework
Servidor Local	PHP Artisan Serve	http://127.0.0.1:8000

3. Estructura de Archivos del Proyecto

3.1 Backend (sistema-monitoreo/)

Ruta / Archivo	Descripción
app/Http/Controllers/AuthController.php	Maneja login y generación de token Sanctum
app/Http/Controllers/ReadingController.php	Recibe lecturas del ESP32 y consulta historial via SP
app/Http/Controllers/UserController.php	CRUD de usuarios/pacientes
app/Models/User.php	Modelo Eloquent con HasApiTokens (Sanctum)
app/Models/Reading.php	Modelo de mediciones de hemoglobina/temperatura
app/Models/Device.php	Modelo de dispositivos ESP32 registrados
app/Models/MedicalAlert.php	Modelo de alertas médicas generadas
app/Models/Role.php	Modelo de roles (Admin=1, Paciente=2)
routes/api.php	Definición de todos los endpoints REST
database/seeder/DatabaseSeeder.php	Datos iniciales: roles, admin, paciente demo, ESP32
database/migrations/	Migraciones de tablas: users, readings, devices, alerts
.env	Variables de entorno: DB, APP_KEY, etc.

3.2 Frontend (sistema-frontend/)

Archivo	Descripción
index.html	Página de login. Llama a POST /api/login y guarda token en localStorage
dashboard.html	Panel principal. Vista Admin (CRUD usuarios) o Vista Paciente (historial)
estilos.css	Hoja de estilos global del frontend

4. Base de Datos

4.1 Esquema de Tablas

Tabla	Campos Clave	Descripción
roles	id, name	Almacena Admin (1) y Paciente (2)
users	id, role_id, full_name, email, password, gender	Usuarios del sistema con contraseña hasheada (bcrypt)
devices	id, board_name, mac_address	ESP32 registrados con su dirección MAC
readings	id, user_id, device_id, hemoglobin_level, temperature, registered_at	Lecturas de sensores almacenadas
medical_alerts	id, reading_id, alert_message, is_critical	Alertas generadas automáticamente por triggers
personal_access_tokens	tokenable_id, token, abilities, expires_at	Tokens Sanctum para autenticación

4.2 Procedimientos Almacenados

sp_get_patient_history(id_usuario)

Retorna el historial clínico completo de un paciente específico con sus alertas médicas.

Llamado desde ReadingController::verHistorialPaciente().

Columnas devueltas: Paciente, Hemoglobina, Temperatura, Fecha_Hora, Alerta_Medica.

sp_get_critical_alerts()

Retorna todas las alertas marcadas como críticas (is_critical = true) del sistema. Disponible en el endpoint GET /api/alertas/criticas.

4.3 Triggers de Base de Datos

Trigger	Acción
Trigger 1	Al insertar el primer usuario, lo promueve automáticamente a Administrador (role_id = 1)
Trigger 4	Al eliminar un usuario, borra en cascada sus medical_alerts y readings antes de eliminar el user

5. API REST - Endpoints

5.1 Rutas Públicas (sin token)

Método	Ruta	Descripción
POST	/api/login	Autenticación. Body: {email, password}. Retorna token + datos usuario
POST	/api/enviar-lectura	Para el ESP32. Body: {mac_address, user_id, hemoglobin_level, temperature}
POST	/api/usuarios/registr	Crear nuevo paciente. Body: {nombre_completo, correo, contrasena, genero}

5.2 Rutas Protegidas (requieren Bearer Token)

Método	Ruta	Descripción
GET	/api/historial?id_usuario={id}	Historial de mediciones del paciente via SP1
GET	/api/usuarios/listar	Lista todos los usuarios registrados (solo Admin)
GET	/api/alertas/criticas	Alertas críticas globales via SP2
POST	/api/usuarios/eliminar	Eliminar usuario. Body: {id_usuario}. Activa Trigger 4

Nota: La ruta PUT /api/usuarios/actualizar no figura en api.php en la versión actual; la actualización se enruta también por POST desde el frontend.

6. Autenticación con Laravel Sanctum

El sistema utiliza Laravel Sanctum para autenticación stateless mediante tokens Bearer.

6.1 Flujo de Autenticación

1. El cliente envía POST /api/login con email y password.
2. AuthController verifica credenciales con Auth::attempt().
3. Si son válidas, genera un token con \$usuario->createToken('TokenMovilWeb').
4. El token (plainTextToken) se devuelve al cliente junto con los datos del usuario.
5. El frontend almacena el token en localStorage como 'token_sanctum'.
6. Las solicitudes posteriores incluyen el header: Authorization: Bearer {token}.

6.2 Control de Roles en Frontend

El dashboard.html detecta el rol del usuario por su ID almacenado (localStorage 'id_usuario'):

- id_usuario === '1' → Vista Administrador (gestión de pacientes, CRUD completo).
- Cualquier otro ID → Vista Paciente (solo lectura de su historial personal).

Nota técnica: El rol se determina en el frontend comparando el ID contra el valor '1'. Una mejora futura sería incluir el campo role en la respuesta del login y verificarlo en el backend.

7. Flujo de Datos del Dispositivo ESP32

7. El ESP32 toma la medición de los sensores (hemoglobina y temperatura).
8. Realiza un POST a /api/enviar-lectura con su MAC address y los valores medidos.
9. ReadingController valida que la MAC exista en la tabla devices. Si no existe, rechaza con 401.
10. Si es válido, crea un nuevo registro en la tabla readings.
11. Los triggers de MySQL procesan la lectura y generan alertas en medical_alerts si corresponde.
12. El médico/admin puede consultar el historial y alertas via el dashboard web.

8. Guía de Instalación y Configuración

8.1 Requisitos del Servidor

- PHP 8.x con extensiones: pdo_mysql, mbstring, openssl, tokenizer
- Composer (gestor de dependencias PHP)
- MySQL 5.7 o superior
- Node.js (opcional, para assets)

8.2 Pasos de Instalación del Backend

13. Clonar o descomprimir el proyecto en el servidor.
14. Ejecutar: composer install en la carpeta sistema-monitoreo/.
15. Copiar .env.example a .env y configurar las variables de entorno:

```
DB_DATABASE=monitoreo_medico_db
DB_USERNAME=root
DB_PASSWORD=tu_password
```

16. Generar la clave de la aplicación: `php artisan key:generate`
17. Crear la base de datos MySQL: `monitoreo_medico_db`
18. Ejecutar migraciones: `php artisan migrate`
19. Ejecutar el seeder de datos iniciales: `php artisan db:seed`
20. Crear los Stored Procedures y Triggers en phpMyAdmin o MySQL Workbench.
21. Iniciar el servidor: `php artisan serve`

8.3 Configuración del Frontend

El frontend es estático y no requiere servidor adicional. Solo necesita que la API esté corriendo en `http://127.0.0.1:8000`. Abrir `index.html` directamente en el navegador o servirlo con cualquier servidor web estático.

8.4 Datos Iniciales (Seeder)

Tipo	Email / MAC	Contraseña
Administrador	admin@correo.com	123456
Paciente Demo	paciente@correo.com	123456
ESP32 Demo	MAC: 24:0A:C4:8D:1B:3C	N/A

9. Seguridad

- Contraseñas hasheadas con `bcrypt` mediante `Hash::make()` de Laravel.
- Autenticación por token Bearer (Sanctum) en todos los endpoints protegidos.
- Validación de dispositivos ESP32 por MAC address antes de aceptar datos.
- El administrador (`id=1`) está protegido contra eliminación en el frontend (botón deshabilitado).
- CORS configurado en `config/cors.php` para controlar orígenes permitidos.

Recomendación: En producción, configurar `APP_DEBUG=false` y HTTPS obligatorio. No exponer `.env` ni credenciales en repositorios públicos.

10. Clasificación de Alertas Médicas

Condición	Rango Hemoglobina	Alerta Generada
-----------	-------------------	-----------------

Anemia (Mujeres)	< 12.0 g/dL	Alerta: Anemia detectada
Anemia (Hombres)	< 13.5 g/dL	Alerta: Anemia detectada
Normal	12.0 - 17.5 g/dL	Sin anomalías (Sano)
Poliglobulia (Mujeres)	> 15.5 g/dL	Alerta: Poliglobulia detectada
Poliglobulia (Hombres)	> 17.5 g/dL	Alerta: Poliglobulia detectada

Nota: Los rangos exactos son definidos por los triggers en la base de datos MySQL.

11. Mantenimiento y Soporte

11.1 Logs del Sistema

Los logs de Laravel se almacenan en storage/logs/laravel.log. Nivel configurado en .env: LOG_LEVEL=debug.

11.2 Actualizar Dependencias

```
composer update
php artisan migrate --force
```

11.3 Agregar Nuevo Dispositivo ESP32

Insertar en la tabla devices la dirección MAC del nuevo ESP32. Puede hacerse directamente en MySQL o creando un endpoint de administración.